Bit-Tech (Binary Digital - Technology)
Vol.9, No.1, August 2026
Available online at: http://jurnal.kdi.or.id/index.php/bt

Automated Black-Box and Manual Basis Path Testing on E-OBE Application

Abdul Azis Naufal Farizqi[1)], Made Hanindia Prami Swari[2)*], I Gede Susrama Mas Diyasa[3)]

[1)2)3)] *Informatics Study Program, Faculty of Computer Science, Universitas Pembangunan Nasional Veteran Jawa Timur, Surabaya, Indonesia*

[1)]21081010210@student.upnjatim.ac.id

[2)]madehanindia.fik@upnjatim.ac.id

[3)]igsusrama.if@upnjatim.ac.id

Article history:

Received 14 Jul 2026;
Revised 23 Jul 2026;
Accepted 03 Aug 2026;
Available online 10 August 2026

Keywords:

Black Box Testing
E-OBE
EvoMaster
REST API
White Box Testing

Abstract

In the current digital era, the development of academic mobile applications, such as the E-OBE application used by Universitas Trunojoyo Madura, requires rigorous testing to ensure robustness and reliability. However, manual testing is often inefficient for detecting complex backend issues and input-handling weaknesses within modern REST API architectures. Objectives: This study aims to evaluate the structural complexity and external robustness of the E-OBE mobile application's backend using a combined testing approach. Methods: The research utilizes White-Box testing, implementing the basis path technique to manually measure the cyclomatic complexity of selected system controllers. Simultaneously, automated Black-Box testing is executed via EvoMaster, targeting REST API endpoints configured with an OpenAPI schema to generate extensive test cases. Results: The White-Box testing revealed that the cyclomatic complexity of the evaluated controller functions ranges from 1 to 5, indicating a relatively simple control flow within the examined source code. Conversely, the automated Black-Box testing (where the optimal run generated 49 HTTP calls and 47 JUnit tests) detected 27 distinct fault manifestations. These unique faults were predominantly API response schema mismatches (22 occurrences), server errors (HTTP 500) (4 occurrences), and an authentication issue (1 occurrence). Conclusions: The evaluated controller functions of the E-OBE application exhibit simple structural complexity. However, the automated testing revealed significant input-handling weaknesses and schema inconsistencies. The API documentation and external input validation require significant improvements to resolve mismatches and prevent server crashes, ensuring better reliability for future front-end integration.

I. INTRODUCTION

In today's era of rapid digital transformation, the integration of information technology into higher education has become a necessity to enhance the quality of academic services [1]. Educational institutions are increasingly adopting digital platforms to manage academic activities, particularly through interconnected mobile and web-based applications [2]. One prominent example is the E-OBE mobile application implemented at Universitas Trunojoyo Madura (UTM). This application was specifically designed to facilitate the assessment and management of the Outcome-Based Education (OBE) curriculum, which includes knowledge, skills, and attitudes. From a technical architecture perspective, the mobile application is supported by a backend system developed in PHP with the Laravel framework, and it uses MySQL for database management. It operationally supports critical academic processes such as curriculum management, semester learning plan (RPS) configurations, and student evaluations managed across distinct user roles, including administrators, curriculum teams, lecturers, and students, orchestrating data exchange across 20 active REST API endpoints. The transition from traditional manual assessment methods to a fully digital OBE system requires a highly reliable and user-friendly platform. Therefore, comprehensively evaluating the quality and performance of the E-OBE mobile application is essential for the academic community to achieve its educational objectives efficiently without technical interruptions [3].

To evaluate the reliability, and optimal functionality of modern mobile applications, rigorous software testing serves as a fundamental phase within the software development lifecycle [4]. The E-OBE mobile application

* Corresponding author

ISSN 2622-2728 (online) 2622-271X (print) © 2026 The Authors. Published by Komunitas Dosen Indonesia.
This is an open access article under the CC BY-SA 4.0 license (http://creativecommons.org/licenses/by/4.0/)
doi: 10.32877/bt.v9i1.4098

utilizes on a Representational State Transfer Application Programming Interface (REST API) architecture to facilitate seamless data communication between the mobile client and the backend server. While REST APIs offer significant advantages in terms of bandwidth efficiency and system interoperability, they also introduce complex integration points that are highly susceptible to unseen structural errors or schema mismatches [5]. According to previous studies on software quality assurance, comprehensive testing of mobile applications is fundamental to ensure that the system operates optimally without backend defects that could disrupt the user experience or compromise academic data integrity [6]. Manual testing alone is generally insufficient to cover the vast array of potential HTTP scenarios and unexpected input variations inherent in RESTful architectures, necessitating the adoption of more advanced and automated testing methodologies [7].

Software testing is theoretically categorized into two complementary approaches: Black-Box and White-Box testing [8]. Black-Box testing evaluates external functionalities based on system inputs and outputs without examining the internal source code [9]. To significantly enhance fault-detection in modern architectures, automated tools like EvoMaster are highly recommended [10]. This tool utilizes evolutionary algorithms to automatically generate system-level test cases for Web APIs based on OpenAPI schemas [11]. Conversely, White-Box testing structurally examines the internal source code and procedural flow to ensure all logical conditions are met [12]. By employing the basis path technique, developers can mathematically measure cyclomatic complexity and identify independent execution paths, ensuring highly effective structural coverage [13].

Several previous studies have explored the development and testing of academic information systems, highlighting various implementation methodologies and their respective limitations. For instance, Magfira et al. developed and advanced an OBE-based curriculum management system using the Model-View-Controller (MVC) architecture and Laravel framework. While their extensive study successfully demonstrated the practical benefits of digital curriculum management and improved collaboration among lecturers, it focused predominantly on early system design and lacked a rigorous, automated testing framework to validate its backend reliability [14]. Basis path testing is a white-box technique used to verify the control structure of a program through a set of independent paths in a control flow graph, and several studies have reported a 100% success rate for generating a complete basis path set on complex graphs [15]. Furthermore, Arcuri comprehensively evaluated automated Black-Box and White-Box testing on RESTful APIs using EvoMaster, firmly concluding that while White-Box testing is superior in code coverage, a synergistic automated testing approach significantly improves overall bug detection capabilities compared to traditional manual methods [11]. Another empirical study by Kim et al. evaluated various automated testing tools, revealing that while tools like EvoMaster are exceptional at triggering server failures (HTTP 500), they still face challenges in achieving high code coverage due to parameter format constraints [16].

Despite extensive research in software testing, a significant gap remains in comprehensively testing mobile-based OBE platforms utilizing REST API architectures. Previous studies often treat development and testing separately or rely on inefficient manual usability testing, which is insufficient for dynamic REST APIs. To bridge this gap, this study presents a case-based application of complementary testing techniques for the E-OBE mobile application by combining manual White-Box testing (basis path technique) to verify internal logical structures and automated Black-Box testing (using EvoMaster) to evaluate external API functionalities. This synergistic paradigm effectively overcomes the limitations of traditional manual testing.

Therefore, this study aims to evaluate system quality, identify robustness defects, and analyze functional performance. Specific objectives include measuring the logical complexity of system controllers and proactively detecting critical API faults—such as schema mismatches, unhandled exceptions, and input-handling weaknesses—to provide actionable architectural recommendations. The paper is organized as follows: Section 2 reviews related literature, Section 3 details the methodology, Section 4 presents the results and discussions, and Section 5 concludes the study.

II. Literature Review

Outcome-Based Education (OBE) is an educational approach that emphasizes the attainment of clear, measurable learning outcomes, with curriculum, instruction, and assessment aligned to develop students' knowledge, skills, and attitudes relevant to professional practice [17]. Unlike traditional educational methods that emphasize the teaching process, OBE is structured around measurable learning outcomes and objective assessment to ensure that students attain the required competencies by graduation [18]. To support this educational framework digitally, higher education institutions rely on mobile applications integrated with Representational State Transfer Application Programming Interfaces (REST APIs). REST APIs provide a lightweight, scalable architectural style that utilizes standardized HTTP protocols for efficient data exchange between mobile clients and backend servers [19]. However, the complex integration of REST APIs in academic systems necessitates software testing to support reliable interoperability and prevent functional failures during academic assessments [20].

Software testing serves as a verification and validation phase within the software development lifecycle, designed to identify defects and ensure that the final product meets its predefined specifications [21]. Theoretically, software quality assurance is divided into two primary complementary methodologies, namely

White-Box testing and Black-Box testing [22]. White-Box testing, frequently referred to as structural testing, involves an examination of the internal source code and procedural logic of the target software [23]. By employing structural techniques such as basis path testing, developers can construct flowgraphs and measure cyclomatic complexity. This mathematical measurement determines the minimum number of independent execution paths, aiming to ensure that internal logical conditions are executed and validated [24]. Conversely, Black-Box testing evaluates the external functionality of the software by examining various inputs and corresponding outputs without any prior knowledge of the internal code architecture [25].

Recent studies implement White-Box testing to evaluate web systems. For instance, Prasetya and Nuryasin applied the basis path technique on a digital marketing platform. To manage code complexity, they built flowgraphs using system-level flowcharts instead of source code. By calculating cyclomatic complexity and defining independent paths, all test cases were executed successfully, verifying the internal procedural logic [26]. While this study demonstrated the utility of basis path testing in identifying logical flaws, its application was limited to traditional monolithic web applications and did not evaluate distributed API functionalities. Consequently, a knowledge gap remains regarding how basis path testing performs when dealing with distributed RESTful architectures.

To address the complexities of modern distributed architectures, recent research has shifted towards specification-based testing and automated Black-Box tools designed for REST APIs. Arcuri evaluated the performance of automated Black-Box and White-Box testing on RESTful APIs utilizing EvoMaster, an open-source tool that utilizes evolutionary algorithms to automatically generate system-level test cases based on OpenAPI or Swagger schemas [10]. The empirical study concluded that while White-Box approaches offer higher internal code coverage, integrating Search-Based Software Testing (SBST) tools like EvoMaster is crucial for detecting unexpected backend bugs and unhandled external exceptions [11]. Furthermore, an empirical study by Kim et al. evaluated various automated API testing tools, including EvoMaster, across multiple real-world open-source web services. Their findings highlighted that although these automated fuzzers are capable of triggering severe server crashes (HTTP 500 errors), they often struggle to achieve high code coverage due to parameter format constraints [16]. In modern API testing, evaluating schema validation and handling issues such as test flakiness or stateful API dependencies remains a critical challenge, making the integration of automated structural and functional tools a necessity.

Despite extensive literature, a critical gap remains in evaluating REST API based academic mobile applications using combined white-box and black-box testing techniques. This research addresses this specific gap by concurrently implementing the manual basis path technique and automated EvoMaster fuzzing to assess the E-OBE application. Acknowledging inherent limitations, such as stochastic search behaviors, OpenAPI schema dependencies, and database state management challenges, this integrated approach thoroughly evaluates both internal procedural logic and external input handling. Ultimately, this provides a comprehensive empirical framework to enhance overall academic platform reliability.

III. METHODS

The research methodology is systematically structured into several chronological phases to evaluate the E-OBE mobile application. The testing procedure begins with a literature review to establish the theoretical foundation, followed by a requirement analysis and problem identification phase. Subsequently, the core execution involves designing and implementing test scenarios, specifically divided into automated Black-Box testing for the RESTful API and manual White-Box testing for the internal application controllers. To visually represent these sequential research steps, a research flowchart was formulated to guide the testing procedures, as illustrated in Figure 1.

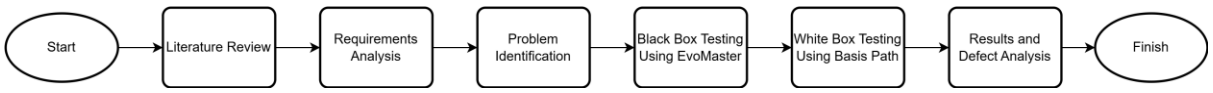


Fig. 1 Research methodology flowchart

During the initial phase, the literature review gathers foundational concepts from academic databases to establish a theoretical framework for the study. This step involves an exploration of Outcome-Based Education (OBE) paradigms, RESTful API architectures, and modern software testing methodologies. By reviewing prior studies, this phase identifies existing research gaps particularly the lack of structural and functional testing in academic mobile platforms. Establishing this theoretical baseline ensures that the subsequent testing procedures are academically grounded and directly aligned with the research objectives.

Requirements Analysis involves understanding the primary objectives and operational needs of the system through direct coordination and requirement gathering with the institution [27]. Once the primary requirements for developing the E-OBE application were identified, they served as a reference during the application testing process. To gain an understanding of the core needs and objectives behind the creation of the E-OBE application,

an interview process was conducted. Through these interviews, the testing was executed in a structured manner in alignment with the research objectives. Ultimately, the tester ascertained whether the application operates in accordance with the primary requirements and the objectives expected by the institution.

Problem identification is an initial step in research that serves as the impetus for conducting the study and determining its internal priorities [28]. The process began by acquiring the source code from the relevant institution. Subsequently, problem identification was conducted to discover potential robustness defects within the system under test. The E-OBE application is evaluated to determine whether its functionality aligns with expectations and whether the available features operate correctly. Rather than selecting only a few, all application controllers including features such as user authentication, course management, MBKM (Merdeka Belajar Kampus Merdeka) management, student evaluation, and Semester Learning Plan (RPS) evaluation are comprehensively evaluated to assess the system's overall quality.

For the external functionality evaluation, the Black Box Testing phase was executed automatically utilizing EvoMaster version 4.0, an open-source Search-Based Software Testing (SBST) tool driven by the Many Independent Objective (MIO) evolutionary algorithm [10]. This specific tool was selected due to its proven empirical effectiveness in automating REST API fault detection and unhandled exception discovery compared to manual black-box methods [11].The chronological process initiated with the manual creation of an OpenAPI/Swagger schema that structurally defined all the REST API endpoints of the E-OBE application. Following the schema generation, the testing environment was configured through a configuration file (em.yaml) to establish authentication protocols, set the execution time limit to exactly one hour per test iteration, and define the maximum HTTP request rate to 30 requests per minute. Crucially, prior to each of the 10 independent testing iterations, the backend database was systematically reset and populated utilizing the Laravel database seeder (php artisan db:seed) to ensure a controlled database state, while EvoMaster was executed utilizing its default random seed to allow stochastic input generation**.** Once configured, the automated tool generated system-level test cases formatted in Java JUnit 4. From the 10 independent iterations conducted, the most optimal run was determined post-hoc based on the highest total of generated HTTP calls and the tool's maximum capability in detecting various HTTP response status codes (including 2xx, 3xx, 4xx, and 5xx). The outputs documented in the EvoMaster REST Report were subsequently analyzed to evaluate the HTTP response codes, and duplicate error manifestations were consolidated based on their specific fault codes to explicitly identify distinct fault types, such as server crashes or schema mismatches, adhering to the standard fault deduplication protocols established in automated API testing research [29].

Conversely, the White Box Testing phase assesses the internal procedural logic of the application's controllers by implementing the basis path technique. This structural technique is deliberately utilized because it provides a rigorous mathematical foundation to guarantee that every independent execution path is systematically evaluated [13]. The execution involves analyzing the PHP source code of each controller function and translating its logical structure into a flowchart. This flowchart is subsequently transformed into a directed flowgraph, which represents the control flow utilizing interconnected nodes, predicate nodes, and edges. By analyzing this flowgraph, the cyclomatic complexity is calculated using the formula V(G) = E - N + 2 (where E represents the total number of edges and N represents the total number of nodes) to determine the minimum number of independent execution paths [30]. Ultimately, these calculated independent paths dictate the required number of test cases and evaluate the internal logical risk level [24], utilizing the cyclomatic complexity and risk relationship table presented in Table 1. Due to space constraints, while a representative modelling example is presented in Chapter IV, the complete artefacts for all evaluated controllers including source code fragments, flowcharts, and flowgraphs are provided in an external supplementary material repository accessible at https://drive.google.com/file/d/1xu74VhtXnWfSvDsGw7MXCOsfYmD5Y6Jp/view?usp=sharing.

TABLE 1
RELATIONSHIP BETWEEN CYCLOMATIC COMPLEXITY AND RISK LEVEL

CC Value	Procedure Type	Risk Level
1 - 4	Simple procedure	Low
5 - 10	Well structured and stable procedure	Low
11 - 20	More complex procedure	Moderate
21 - 50	Complex and critical procedure	High
>50	Error prone, highly disruptive, unstable procedure	Very High

Following the execution of both automated and manual testing procedures, the methodology culminates in the Results and Defect Analysis phase. In this stage, the empirical data gathered from the EvoMaster HTTP fuzzing and the calculated cyclomatic complexity metrics are consolidated. The identified functional anomalies, particularly API response schema mismatches and backend server crashes, are cross-referenced against the verified internal logical structures. This analytical process aims to isolate the root causes of the robustness defects, distinguishing between external input validation failures and internal procedural limitations. Ultimately, this defect analysis synthesizes actionable architectural recommendations to enhance the backend reliability of the E-OBE application, thereby fulfilling the research objectives and concluding the methodological workflow.

IV. RESULTS

The performance evaluation and quality assurance of the E-OBE mobile application were rigorously conducted through two complementary testing methodologies. The White-Box testing phase meticulously evaluated the internal procedural logic of the E-OBE application by mathematically analyzing the underlying PHP source code of the system controllers. To illustrate this methodological procedure, the destroy function within the Evaluasi controller is presented as a representative modeling example. The original PHP source code for this function is shown in Table 2.

TABLE 2
SOURCE CODE OF DESTROY FUNCTION EVALUATION CONTROLLER

No	Source Code
1	public function destroy($id_evaluasi)
2	{
3	$result = $this->evalRepositoryInterface->delete($id_evaluasi);
4	if ($result) {
5	return ApiResponseClass::sendResponse('true', 'Evaluasi Delete Success', 200);
6	} else {
7	return ApiResponseClass::sendResponse('false', 'Evaluasi Not Found', 404);
8	}
9	}

To systematically measure its complexity, the control flow of the source code is first mapped into a flowchart and subsequently transformed into a directed flowgraph, as depicted in Figure 3 and Figure 4.

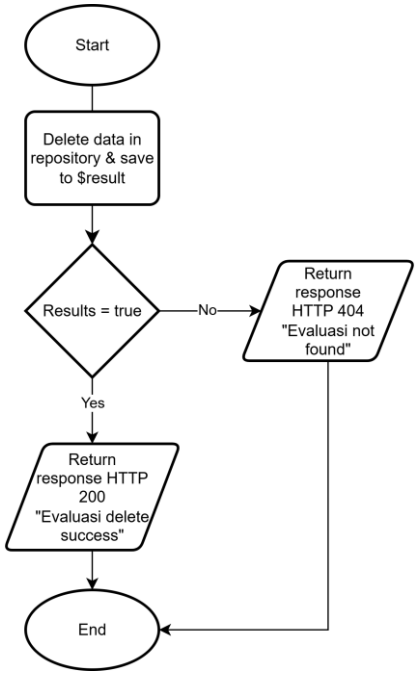


Fig. 3 Flowchart of the evaluation destroy function

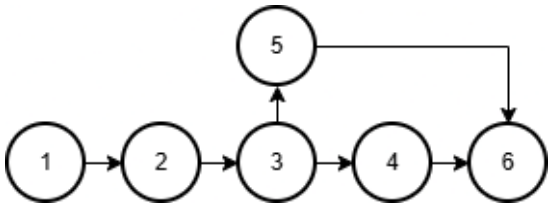


Fig. 4 Flowgraph of the evaluation destroy function

Based on the constructed flowgraph, the structure consists of 6 nodes (N) and 6 edges (E). Using the McCabe cyclomatic complexity formula, the risk level is mathematically calculated as follows: V(G) = E - N + 2 V(G) = 6 - 6 + 2 V(G) = 2. The calculation yields a Cyclomatic Complexity value of 2, indicating that the destroy function possesses exactly two independent execution paths. This identical structural modeling and mathematical calculation procedure was systematically applied to all other evaluated system controllers. The complete summary of the Cyclomatic Complexity calculations and their corresponding risk levels for all controllers is presented in Table 3.

TABLE 3
SUMMARY OF CYCLOMATIC COMPLEXITY AND RISK LEVELS

Controller Module	Evaluated Functions	Cyclomatic Complexity	Risk Level
Auth	Login	3	Low
Halaman Utama	getNamaDosen, getProfilDosen	2	Low
PengampuMk	Show, showMatkul	2	Low
DetailRPS	showMinggu		
Evaluasi	showEvaluasi, store, update, destroy	2	Low
Evaluasi	getDropDown	1	Low
EvaluasiMahasiswaData	showByMatkul	2	Low
EvaluasiMahasiswaDetail	getByUserId, store, update, destroy	2	Low
EvaluasiMahasiswaDetail	calculateEvaluasi	5	Low
MatkulCpl	showByUserId	1	Low
Cpl	show	2	Low

The results consistently yielded impressively low values, strictly ranging from one to five. Fundamental functions such as single data retrieval (e.g., retrieving dropdown data or weekly schedules) frequently scored a complexity of one or two, indicating a straightforward, linear execution path. In contrast, more intricate algorithmic operations, such as the calculateEvaluasi function within the student assessment module, achieved a maximum complexity score of five due to multiple conditional branches required for calculating and validating grading weights. According to standard software engineering risk metrics, these specific complexity values explicitly confirm that the internal source code falls strictly into the simple and well-structured category, inherently possessing a remarkably low risk of internal logical failure. Consequently, all the systematically derived independent paths were successfully executed during the manual testing phase, resulting in a flawless validation rate where every actual output perfectly matched the expected logical outcome.

Conversely, the automated Black-Box testing, executed via the EvoMaster testing tool, was performed iteratively over ten separate sessions with a strictly configured maximum execution time of one hour per iteration. A detailed summary of the execution results, including the total HTTP calls, #2xx, #3xx, #4xx, #5xx, and distinct fault types for each iteration, is systematically presented in Table 4.

TABLE 4
SUMMARY OF EVOMASTER AUTOMATED TESTING EXECUTIONS

Run ID	HTTP CALLS	#2xx	#3xx	#4xx	#5xx	Distinct Fault Types
1	48	16	0	20	4	3
2	48	16	0	20	4	3
3	48	16	0	20	4	3
4	48	16	0	20	4	3
5	48	16	0	20	4	3
6	48	16	0	20	4	3
7	49	16	0	20	4	3
8	48	16	0	20	4	3
9	49	16	0	20	4	3
10	47	16	0	20	4	3

Among these comprehensive test executions, the optimal testing run successfully generated 49 HTTP calls and automatically produced 47 distinct test cases formulated in the Java JUnit 4 format. These automatically generated test cases aggressively traversed various REST API endpoints, successfully uncovering critical backend robustness defects and unexpected functional behaviors that traditional manual testing procedures had previously failed to identify. The overall execution summary and the proportion of HTTP response codes generated by the tool are visually represented in Figure 5 and the total generated files and faults summary are detailed in Figure 6.

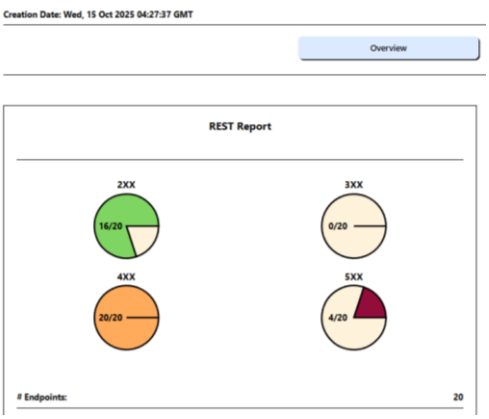


Fig. 5 REST Report black box testing EvoMaster

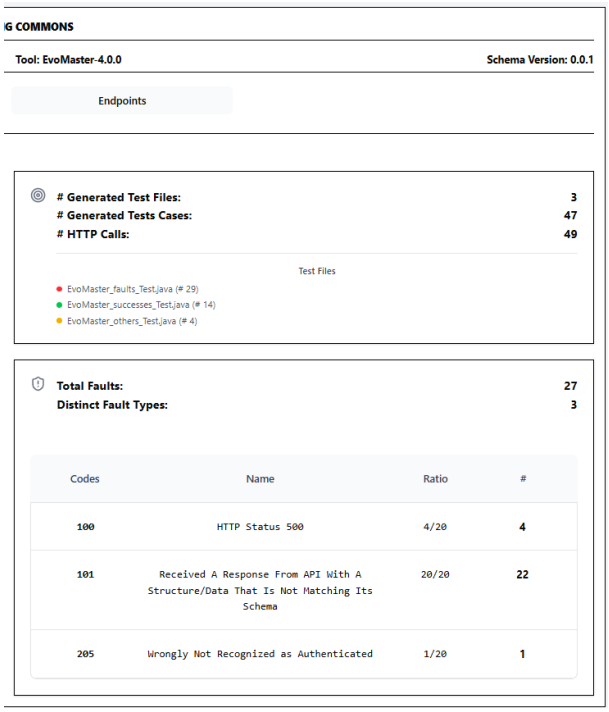


Fig. 6 Webreport black box testing EvoMaster

From the comprehensive execution of the automated test cases, the Black-Box testing proactively identified 27 distinct system faults out of the 49 generated HTTP requests. As systematically summarized in Table 5, these identified faults were explicitly categorized into three distinct fault types based on their underlying architectural triggers

TABLE 5
DISTINCT FAULT TYPES DETECTED BY EVOMASTER TESTING

Fault code	Fault Category Name	Total Occurrences
100	HTTP 500 (Internal Server Error)	4
101	API Response Mismatch	22
205	Authentication Issue / Unrecognized Token	1

The most predominant fault was the API response schema mismatch (Fault Code 101), which occurred 22 times across various endpoints. This high occurrence unequivocally indicates that the actual server responses frequently deviated from the officially documented OpenAPI schema, such as returning undocumented 401 (Unauthorized) or 500 (Internal Server Error) status codes, or returning array instances when the schema expected string primitives. Furthermore, the testing framework successfully triggered four critical server crashes (Fault Code 100 or HTTP 500) within the authentication and student evaluation modules (e.g., during store, update, and delete operations). These crashes were predominantly caused by unhandled exceptions, such as the system's failure to validate an “Undefined array key email” during the login process. Lastly, the tool discovered one specific authentication inconsistency (Fault Code 205) where a valid user login was improperly rejected by subsequent endpoints.

V. DISCUSSION

The evaluation of the E-OBE mobile application using both White-Box and automated Black-Box testing methodologies provided complementary insights into the system's structural complexity and functional robustness. Based on the manual White-Box testing utilizing the basis path technique, the calculated Cyclomatic Complexity for the evaluated controllers consistently ranged from one to five. While this low complexity score indicates that the internal control flow of the selected functions is relatively simple, it should not be interpreted as essential evidence of a highly efficient, secure, or maintainable overall architecture. Software quality and failure risk must ideally be assessed alongside other structural metrics, such as overall code coverage, exception handling mechanisms, and defect density, before broader architectural conclusions are drawn.

The automated Black-Box testing via EvoMaster uncovered 27 distinct faults from 49 generated HTTP calls, predominantly API response schema mismatches (22 occurrences) and server crashes (4 occurrences). The high frequency of schema mismatches necessitates critical consideration; because the OpenAPI schema was manually constructed for this study, these mismatches may reflect inaccuracies or omissions in the manually authored specification itself rather than solely indicating poor documentation maintenance by the developers. Furthermore,

the detection of HTTP 500 errors such as the "Undefined array key email" during authentication highlights vulnerabilities in input validation. Rather than viewing this as a contradiction between sound internal logic and weak external handling, these crashes suggest that certain internal validation logic, database states, or framework configurations possess unhandled branches that were simply not captured by the selected manual basis paths.

These empirical findings corroborate the conclusions drawn from previous software testing studies while highlighting the limitations of relying on single testing approaches. In contrast to Sasmito's research [26], which reported perfect validation success using solely the basis path technique on a monolithic web application, this study demonstrates that manual structural verification alone is insufficient for fully evaluating dynamic RESTful architectures. The results align with Arcuri's findings [10]. which emphasized that integrating automated algorithms like EvoMaster is critical for detecting unhandled external exceptions and unexpected backend bugs. Additionally, the prevalence of HTTP 500 server crashes triggered by automated input fuzzing directly supports the empirical study by Kim et al [16], which highlighted the effectiveness of automated fuzzers in exposing server vulnerabilities caused by input format constraints.

Finally, an analysis of the experimental stability across the ten independent EvoMaster runs reveals nearly identical outcomes, consistently generating 16 successful responses and 4 server errors per run. This high consistency indicates that the stochastic search algorithm repeatedly encountered the same fragile endpoints particularly within the authentication module without achieving new defect diversity or broader endpoint coverage. This limitation may stem from the one-hour execution time constraint or the fuzzer's inability to generate specific parameter formats to bypass initial blocking exceptions. Future research should prioritize broader endpoint coverage, reproducible experimental configurations, stateful database control, and strict validation of the OpenAPI contract against the implemented API to formulate a more holistic quality assurance paradigm.

VI. CONCLUSIONS

The comprehensive evaluation of the E-OBE mobile application successfully integrated both White-Box and automated Black-Box testing methodologies to explicitly answer the research objectives regarding overall system quality and reliability. Based on the manual White-Box testing utilizing the basis path technique, the mathematically calculated Cyclomatic Complexity values consistently ranged from one to five across all evaluated system controllers. This evidence supports that the selected controller functions possess relatively simple control flows and that their manually derived paths produced expected outputs under the tested conditions. However, automated Black-Box testing utilizing the EvoMaster tool revealed significant response-contract and error-handling defects. The automated fuzzing generated 49 HTTP calls and successfully detected 27 distinct faults, often dominated by API response schema mismatches (22 occurrences) and critical server crashes (4 occurrences).

This discrepancy highlights a critical gap between the internal procedural logic and external input validation mechanisms; the observed defects establish inadequate validation and robustness in selected API operations, leading to unhandled backend exceptions that could severely disrupt digital academic assessment processes. Consequently, the primary contribution of this manuscript is an empirical demonstration that manual structural testing and automated API fuzzing reveal different categories of software-quality evidence in an academic REST API.

REFERENCES

[1] N. Kavitha, T. Anitha, M. L. Chandrika, N. Venugopal, R. Vinupriya, and K. M. Priya, "The Impact of Mobile Applications on the Development of Academic Learning," in *2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS)*, Coimbatore, India: IEEE, Mar. 2024, pp. 2634–2639, https://doi.org/10.1109/ICACCS60874.2024.10717048

[2] F. J. García-Peñalvo, "Avoiding the Dark Side of Digital Transformation in Teaching. An Institutional Reference Framework for eLearning in Higher Education," *Sustainability*, vol. 13, no. 4, p. 2023, Feb. 2021, https://doi.org/10.3390/su13042023

[3] D. Y. Mo, Y. M. Tang, E. Y. Wu, and V. Tang, "Theoretical model of investigating determinants for a successful Electronic Assessment System (EAS) in higher education," *Educ. Inf. Technol.*, vol. 27, no. 9, pp. 12543–12566, Nov. 2022, https://doi.org/10.1007/s10639-022-11098-1

[4] Kusum, Pritika Talwar, Amit Puri, and Guneet Kumar, "Overview of software testing," *Glob. J. Eng. Technol. Adv.*, vol. 19, no. 1, pp. 104–112, Apr. 2024, https://doi.org/10.30574/gjeta.2024.19.1.0060.

[5] K. Habib, M. H. M. Saad, A. Hussain, M. R. Sarker, and K. A. Alaghbari, "An Aggregated Data Integration Approach to the Web and Cloud Platforms through a Modular REST-Based OPC UA Middleware," *Sensors*, vol. 22, no. 5, p. 1952, Mar. 2022, https://doi.org/10.3390/s22051952.

[6] S. W. G. AbuSalim, R. Ibrahim, and J. A. Wahab, "Comparative Analysis of Software Testing Techniques for Mobile Applications," *J. Phys. Conf. Ser.*, vol. 1793, no. 1, p. 012036, Feb. 2021, https://doi.org/10.1088/1742-6596/1793/1/012036.

[7] A. Golmohammadi, M. Zhang, and A. Arcuri, "Testing RESTful APIs: A Survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 1, pp. 1–41, Jan. 2024, https://doi.org/10.1145/3617175.
[8] Z. Chen *et al.*, "Exploring Better Black-Box Test Case Prioritization via Log Analysis," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 3, pp. 1–32, Jul. 2023, https://doi.org/10.1145/3569932.
[9] F. A. Artanto, A. Khambali, S. Nadifa, and V. A. Azarine, "Black Box Testing Dengan Teknik Equivalence Partitioning Pada Aplikasi MJ Autocare," *Digit. Transform. Technol.*, vol. 4, no. 1, pp. 693–698, Aug. 2024, https://doi.org/10.47709/digitech.v4i1.4240.
[10] A. Arcuri *et al.*, "Tool report: EvoMaster—black and white box search-based fuzzing for REST, GraphQL and RPC APIs," *Autom. Softw. Eng.*, vol. 32, no. 1, p. 4, May 2025, https://doi.org/10.1007/s10515-024-00478-1.
[11] A. Arcuri, "Automated Black- and White-Box Testing of RESTful APIs With EvoMaster," *IEEE Softw.*, vol. 38, no. 3, pp. 72–78, May 2021, https://doi.org/10.1109/MS.2020.3013820.
[12] S. D. Semujju, H. Huang, F. Liu, Y. Xiang, and Z. Hao, "Search-Based Software Test Data Generation for Path Coverage Based on a Feedback-Directed Mechanism," *Complex Syst. Model. Simul.*, vol. 3, no. 1, pp. 12–31, Mar. 2023, https://doi.org/10.23919/CSMS.2022.0027.
[13] J. B. L. Sie, Izmy Alwiah Musdar, and Syamsul Bahri, "Pengujian White Box Testing Terhadap Website Room Menggunakan Teknik Basis Path," *KHARISMA Tech*, vol. 17, no. 2, pp. 45–57, Sep. 2022, https://doi.org/10.55645/kharismatech.v17i2.235.
[14] F. Magfira, T. Matulatan, N. F. Fahmitra, F. Irawan, and R. Herikson, "Implementation of Model View Controller Architecture in Designing Outcome-Based Education (OBE) Curriculum Management Information System," *BIO Web Conf.*, vol. 134, p. 05002, 2024, https://doi.org/10.1051/bioconf/202413405002.
[15] S. Agarwal, S. Godboley, and P. R. Krishna, "Cyclomatic Complexity Analysis for Smart Contract Using Control Flow Graph," in *Computing, Communication and Learning*, vol. 1729, S. K. Panda, R. R. Rout, R. C. Sadam, B. V. S. Rayanoothala, K.-C. Li, and R. Buyya, Eds., in Communications in Computer and Information Science, vol. 1729, Cham: Springer Nature Switzerland, 2022, pp. 65–78. https://doi.org/10.1007/978-3-031-21750-0_6.
[16] M. Kim, Q. Xin, S. Sinha, and A. Orso, "Automated test generation for REST APIs: no time to rest yet," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, Virtual South Korea: ACM, Jul. 2022, pp. 289–301. https://doi.org/10.1145/3533767.3534401.
[17] M. M. Syeed, A. Shihavuddin, M. F. Uddin, M. Hasan, and R. H. Khan, "Outcome Based Education (OBE): Defining the Process and Practice for Engineering Education," *IEEE Access*, vol. 10, pp. 119170–119192, 2022, https://doi.org/10.1109/ACCESS.2022.3219477.
[18] H. Hamidi, A. B. Hejran, A. Sarwari, and S. G. Edigeevna, "The Effect of Outcome-Based Education on Behavior of Students," *Eur. J. Theor. Appl. Sci.*, vol. 2, no. 2, pp. 764–773, Mar. 2024, https://doi.org/10.59324/ejtas.2024.2(2).68.
[19] Sireesha Addanki, "Architectural Shifts in API Design: The Progression from SOAP to REST and GraphQL," *Int. J. Sci. Technol.*, vol. 16, no. 2, p. 6579, Jun. 2025, https://doi.org/10.71097/IJSAT.v16.i2.6579.
[20] H. Sartaj, S. Ali, and J. Marie Gjøby, "REST API Testing in DevOps: A Study on an Evolving Healthcare IoT Application," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 6, pp. 1–46, Jun. 2026, https://doi.org/10.1145/3765744.
[21] I. Ozkaya, "Can We Really Achieve Software Quality?," *IEEE Softw.*, vol. 38, no. 3, pp. 3–6, May 2021, https://doi.ieeecomputersociety.org/10.1109/MS.2021.3060552.
[22] A. Martin-Lopez, A. Arcuri, S. Segura, and A. Ruiz-Cortes, "Black-Box and White-Box Test Case Generation for RESTful APIs: Enemies or Allies?," in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*, Wuhan, China: IEEE, Oct. 2021, pp. 231–241. https://doi.org/10.1109/ISSRE52982.2021.00034.
[23] T. Avdeenko and K. Serdyukov, "Automated Test Data Generation Based on a Genetic Algorithm with Maximum Code Coverage and Population Diversity," *Appl. Sci.*, vol. 11, no. 10, p. 4673, May 2021, https://doi.org/10.3390/app11104673.
[24] D. Kafura, "Reflections on McCabe's Cyclomatic Complexity," *IEEE Trans. Softw. Eng.*, vol. 51, no. 3, pp. 700–705, Mar. 2025, https://doi.org/10.1109/TSE.2025.3534580.
[25] H. Wang, Y. Zhang, and X. Fu, "Innovative Applications of Black Box Testing: A New Strategy for Teaching Assessment in Higher Education," *IEEE Access*, vol. 13, pp. 90514–90526, 2025, https://doi.org/10.1109/ACCESS.2025.3568825.
[26] B. Prasetya and I. Nuryasin, "Pengujian Kualitas Website Pt Media Citra Digitalindo Blitar Menggunakan White Box Testing Dengan Teknik Basis Path," *jipi. jurnal. ilmiah. penelitian. dan. pembelajaran. informatika.*, vol. 10, no. 2, pp. 1595–1601, May 2025, https://doi.org/10.29100/jipi.v10i2.7693.

[27] C. Palomares, X. Franch, C. Quer, P. Chatzipetrou, L. López, and T. Gorschek, "The state-of-practice in requirements elicitation: an extended interview study at 12 companies," *Requirements Eng*, vol. 26, no. 2, pp. 273–299, Jun. 2021, doi: https://doi.org/10.1007/s00766-020-00345-x.

[28] M. Alfuth, J. Klemp, A. Schmidt, L. Streese, N. Ramadanov, and R. Prill, "SQUARE-IT: a proposed approach to square the identified research problem in the literature with the objectives, the appropriate clinical research question, and the research hypothesis," *BMC Med Res Methodol*, vol. 25, no. 1, p. 19, Jan. 2025, doi: https://doi.org/10.1186/s12874-025-02468-7.

[29] B. Marculescu, M. Zhang, and A. Arcuri, "On the Faults Found in REST APIs by Automated Test Generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, pp. 1–43, Jul. 2022, doi: https://doi.org/10.1145/3491038.

[30] D. R. Wijendra and K. P. Hewagamage, "Analysis of Cognitive Complexity with Cyclomatic Complexity Metric of Software," *IJCA*, vol. 174, no. 19, pp. 14–19, Feb. 2021, doi: https://doi.org/10.5120/ijca2021921066.